La capa de aplicación

La Web y HTTP (2ª parte).

Tema 2: La capa de aplicación

2.1 Principios de las aplicaciones de red.

2.2 La Web y HTTP (2ª parte).

2.3 Correo electrónico en Internet.

2.4 DNS: el servicio de directorio de Internet.

2.5 Distribución de archivos P2P.

2.6 Flujos de vídeo y redes de distribución de contenidos.

2.7 Programación de sockets: creación de aplicaciones de red.

HTTP

- Interacción usuario-servidor
  - Hay sitios web que necesitan identificar al usuario:
    - porque son de acceso restringido (control de acceso), y/o
    - porque dan un servicio distinto según quién sea.
  - **HTTP no tiene estado**: cada petición llega sin memoria de las anteriores.
  - Dos mecanismos ponen ese estado por encima de HTTP:
    - Autorización (`Authorization`), y
    - Cookies (`Set-Cookie` / `Cookie`).

Autorización HTTP

- Mecanismo "específico" para identificar al usuario, a petición del servidor:
  - código de estado `401 Unauthorized`,
  - con la cabecera `WWW-Authenticate`.
- El navegador repite la petición con las credenciales:
  - en la cabecera `Authorization` (nombre de usuario y clave).
  - Esquema `Basic`: `Authorization: Basic YWx1bW5vOmNsYXZl` es `alumno:clave` en Base64, **no cifrado**: legible en cualquier captura.
  - El usuario las teclea una vez; el navegador las recuerda durante la visita.
  - **Sin estado**: `Authorization` en cada petición posterior a ese servidor.
  - **Con estado**: tras el 200 que contesta a la petición con `Authorization`, el servidor envía `Set-Cookie` de sesión y desde ahí el navegador envía `Cookie` en vez de `Authorization`.

- Cookies HTTP (RFC 6265)
  - Mecanismo "genérico" para identificar al usuario.
  - Identifica a un **navegador concreto** (un perfil de usuario en un equipo), no a la persona.
  - Lo usan la mayoría de los sitios web:
    - para dar un servicio personalizado: sesión iniciada, carrito de la compra, idioma, preferencias…
    - para hacer seguimiento de la actividad del usuario: publicidad, comercio electrónico, analítica (cookies de terceros, más adelante).

- Componentes para el soporte de cookies
  - Cabeceras específicas: `Set-Cookie` en las respuestas, `Cookie` en las peticiones.
  - Almacén de cookies del navegador: una fila (sitio, nombre, valor, caducidad) por cookie. Lo gestiona el navegador.
  - Base de datos en el servidor: el valor de la cookie es solo la **clave**; el estado real (sesión, carrito…) está en el servidor.

- Procedimiento con cookies
  - Primera visita a un sitio que usa cookies. El servidor:
    - Crea un identificador único y una entrada en su base de datos;
    - Responde con la cabecera `Set-Cookie: id=1678` (siempre nombre=valor), **solo en esa primera respuesta**.
    - Atributos opcionales:
      - `Max-Age` o `Expires` (cuánto dura; sin ellos muere al cerrar el navegador)
      - `Domain` y `Path` (a qué peticiones se adjunta)
      - `Secure` (solo por HTTPS), `HttpOnly` (no accesible desde JavaScript).
  - El navegador guarda la fila (sitio, `id=1678`, caducidad) en su almacén.
  - Desde entonces, **todas** las peticiones a ese sitio llevan `Cookie: id=1678`: también las de las imágenes y las de visitas posteriores, mientras no caduque.
  - **Nunca a otro sitio**: el almacén es por sitio; otro servidor no recibe esta cookie.

Ejemplo de procedimiento con cookies

Las cookies proveen cierta información de "estado de sesión" por "encima" de HTTP, que es un protocolo que no maneja estados.

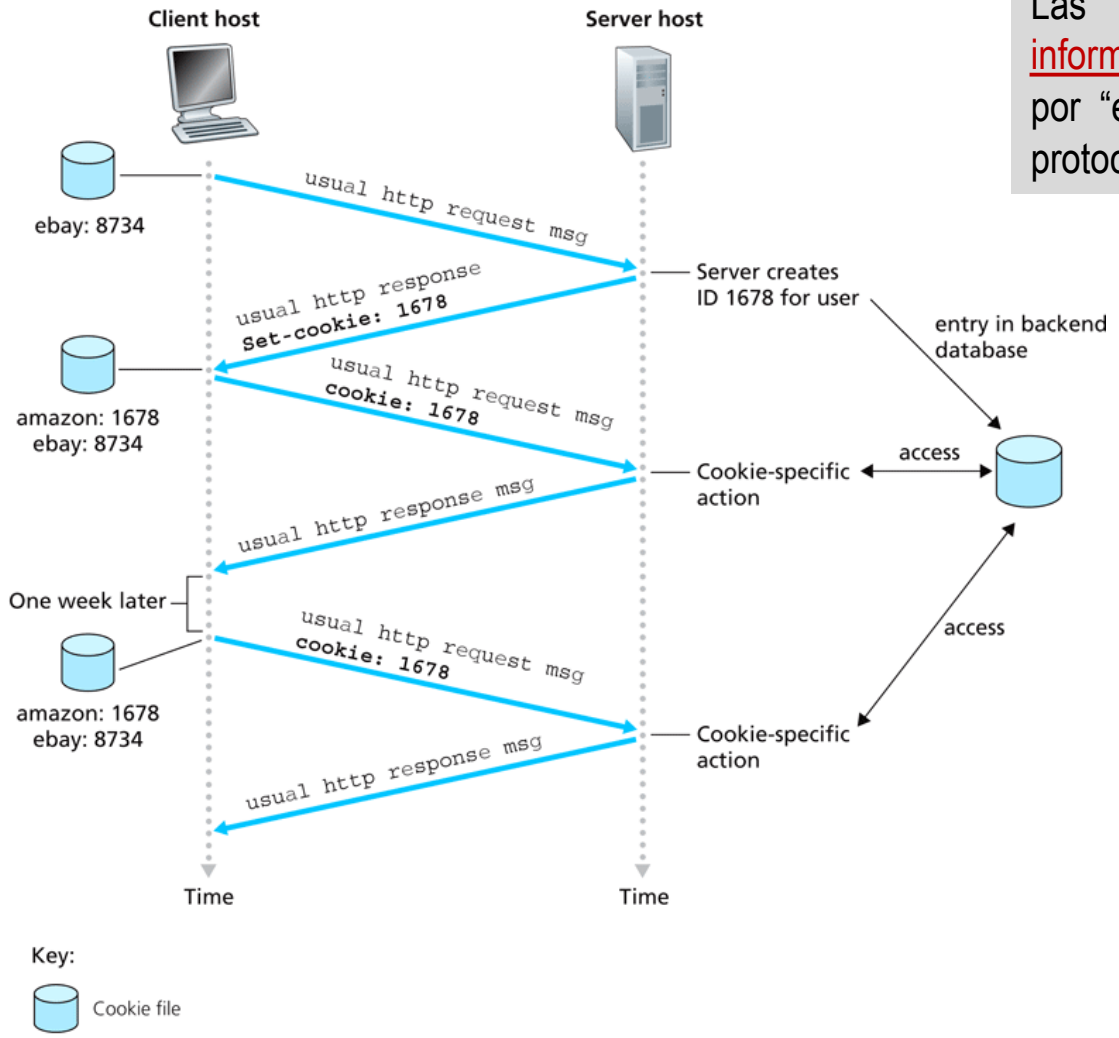


Figure 2.10 ♦ Keeping user state with cookies

- Cookies de terceros: cómo funciona el seguimiento
  - `periodico.es` y `tienda.es` incluyen una imagen (un anuncio) alojada en **ads.ejemplo.net**.
  - Al cargar periodico.es, el navegador pide la imagen a ads.ejemplo.net, que responde con `Set-Cookie: uid=4411`.
  - Al cargar tienda.es, la imagen se pide de nuevo a ads.ejemplo.net, ahora con `Cookie: uid=4411`: es el mismo sitio.
  - ads.ejemplo.net sabe que el mismo navegador ha visitado los dos sitios y, con `Referer`, qué páginas, cuándo y en qué orden.
  - Cookie de terceros = la pone un sitio distinto del que aparece en la barra de direcciones.

- Caché WEB
  - Procedimiento para mejora de las prestaciones del servicio Web. Permite reducir:
    - El tiempo de obtención de un objeto (latencia), en particular para:
      - Objetos grandes y poco ancho de banda (AB) de acceso.
      - Sobre todo si **AB limitante** entre cliente y servidor **< AB limitante** entre cliente y caché Web.
    - El tráfico en la red → Mayor rendimiento para todas las aplicaciones.
      - De manera global y especialmente en los enlaces de acceso a Internet. P.e, en un acceso institucional.
      - Permite aprovechar mejor la tasa del acceso disponible → Reducción de costes.
    - La carga de trabajo de servidores Web.
  - Es almacenamiento en memoria temporal (caché) de copias de objetos Web.
    - En el cliente (lo gestiona el navegador).
    - En un servidor de red, como proxy (intermediario) Web.
  - Problema: actualidad de los objetos en caché.
    - Si un objeto se actualiza/modifica en el servidor, la copia en caché NO es válida.
    - Solución: **GET condicional.**

Los mensajes de respuesta relativos a un objeto suelen contener la cabecera "`Last-Modified`" (si no, `ETag`)

con valor = fecha/hora de creación/actualización del objeto.

GET condicional

- Mecanismo HTTP para comprobar y garantizar la actualidad de los objetos en caché.
- Se implementa mediante mensaje GET:
  - Solicitud condicionada:
    - **Se emite solo si la copia en caché no es fresca** (antigüedad mayor que `max-age` o si se ha indicado `no-cache`)
    - Se pide que se devuelva el objeto sólo si ha sido modificado respecto a la copia en caché.
  - Con cabecera específica "`If-Modified-Since`".
    - `If-Modified-Since=Last-Modified` del objeto que tenemos

Respuesta al GET condicional

- Si el objeto solicitado NO ha sido modificado.
  - Servidor responde con "`304 Not Modified`" y el cuerpo del mensaje vacío → la copia vuelve a ser fresca otro periodo de duración `max-age`
- Si el objeto solicitado ha sido modificado.
  - Servidor responde con "`200 OK`" y el cuerpo del mensaje contiene el objeto solicitado.
    - Cabecera "`Last-Modified`" con un nuevo valor, más reciente: contiene fecha/hora de la última actualización.

- Ejemplo de “GET condicional”.

1er Mensaje de Solicitud (GET normal)

```
GET /fruta/kiwi.gif HTTP/1.0
User-agent: Mozilla/4.0
```

2do Mensaje de Solicitud (GET condicional)

```
GET /fruta/kiwi.gif HTTP/1.0
User-agent: Mozilla/4.0
If-Modified-Since: Mon, 22 Jun 1998 09:23:24
```

1er Mensaje de Respuesta

```
HTTP/1.0 200 OK
Date: Wed, 12 Aug 1998 15:39:29
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 1998 09:23:24
Content-Type: image/gif

( …….contenido del objeto……)
```

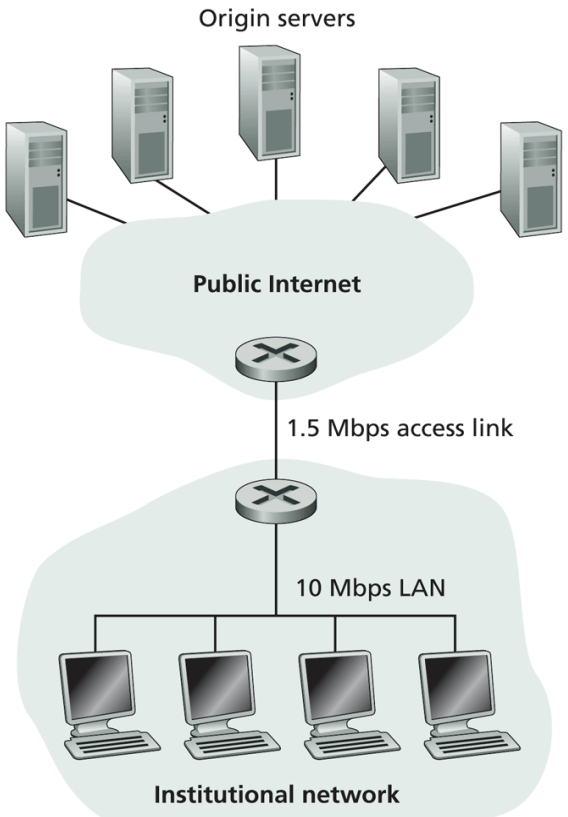
2do Mensaje de Respuesta (objeto válido en caché)

```
HTTP/1.0 304 Not Modified
Date: Wed, 19 Aug 1998 15:06:12
Server: Apache/1.3.0 (Unix)

( ……cuerpo vacío, sin contenido……)
```

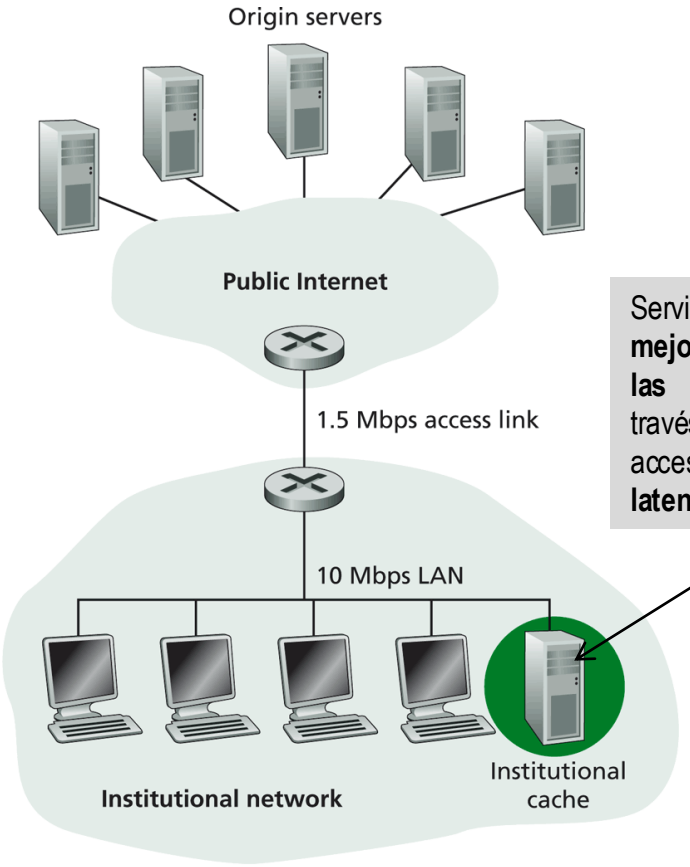
- Proxy: caché web compartida → servidor proxy.
  - Atiende peticiones HTTP en nombre de los servidores web. Doble papel:
    - de cara al navegador actúa como servidor web;
    - de cara al servidor web actúa como cliente web.
  - Guarda en disco copias de los objetos que piden sus usuarios y aplica las mismas reglas que el navegador: frescura (`max-age`) y GET condicional.
  - Tasa de aciertos = aciertos / solicitudes. Crece con el número de usuarios (en la práctica, 20–70 %).
  - Se usa también para control de acceso (quién, qué contenidos, qué protocolos) y para distribuir contenidos.
  - Hoy: **con HTTPS el proxy no ve ni la URL ni el objeto → el proxy caché compartido casi ha desaparecido**.
    - Su papel lo hacen la caché del navegador y las CDN.
    - Sigue en redes corporativas como control de acceso, instalando un certificado propio para inspeccionar TLS.

“Cuello de botella” sin proxy



Datos de las figuras: 15 pet/s × 100 kbit sobre el acceso de 1,5 Mbps ⟶ intensidad 1,0 (retardo enorme). Con 40 % de aciertos ⟶ 0,6; retardo medio ≈ 0,4·(ms) + 0,6·(2 s) ≈ 1,2 s.

Servidor proxy en la Intranet

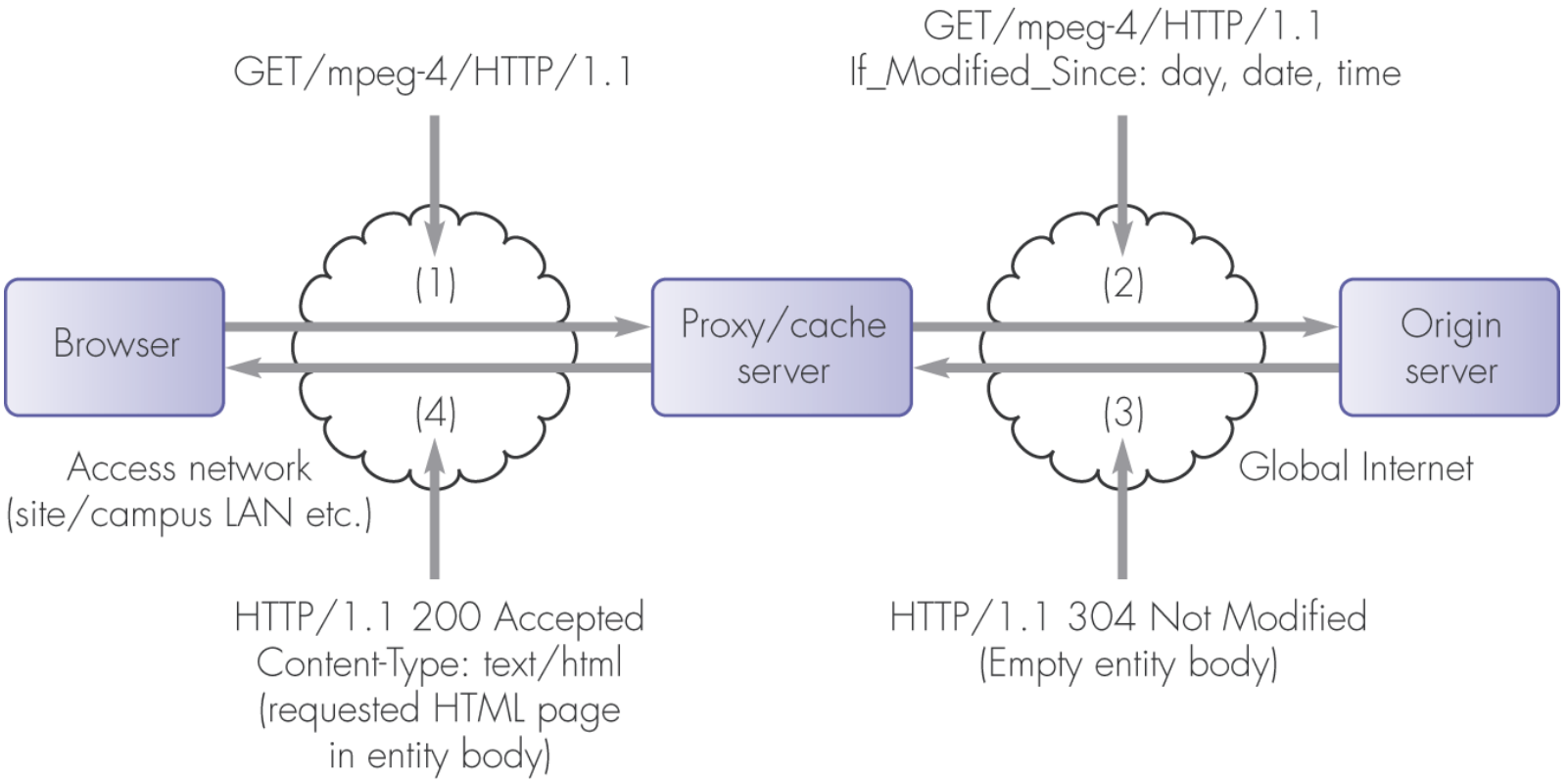


Servidor proxy interno **mejora rendimiento de las comunicaciones** a través del enlace de acceso y **reduce la latencia del servicio**

El servidor proxy en la Intranet requiere:
Interceptación: mecanismo de re-direccionamiento de solicitudes Web a un proxy, o
Configuración del Navegador Web

- ¿Cómo llega mi petición al proxy?
  - **Proxy configurado** en el navegador (nombre o IP y puerto; a mano, por el sistema o por autodetección):
    - el navegador abre la conexión TCP **contra el proxy**, no contra el servidor;
    - la línea de petición lleva la **URL absoluta**: `GET http://www.soluciones.es/index.html HTTP/1.1` (y no solo la ruta `/index.html`);
    - señal en una captura: la IP de destino no es la del `Host`.
  - **Proxy transparente** (interceptación en el router de la red): el navegador no sabe nada; la petición es la normal y el proxy la desvía.
  - Ejercicios:
    - Tenéis ejercicios propuestos de creación de secuencias de HTTP como de análisis de capturas de Wireshark

"GET condicional" con servidor Proxy.



Fuente: Halsall, *Computing Networking and the Internet,* 5th Edition © Pearson Education Limited 2005

- Reglas para la caché web: ¿quién decide?
  - El **servidor**, con cabeceras en cada respuesta:
    - `Cache-Control: max-age=N` → la copia es fresca N segundos desde que se recibió.
    - `Cache-Control: no-cache` → se guarda, pero se revalida siempre (GET condicional).
    - `Cache-Control: no-store` → no se guarda.
    - `Expires: <fecha>` → lo mismo que `max-age`, con fecha absoluta (HTTP/1.0).
    - `Last-Modified` / `ETag` → no dan frescura: sirven para revalidar.
  - Las etiquetas <meta> del HTML no cuentan: las cachés no leen el cuerpo de los mensajes.
  - El usuario del navegador y el administrador del proxy pueden vaciar o desactivar la caché, pero no alargar la frescura.
  - Misma idea que el TTL de los registros DNS: quien publica el dato dice cuánto tiempo vale.

- ¿Caché o servidor? Decisión para cada objeto
  - No se guarda si la respuesta trae `no-store` (ni, en general, si no trae información de frescura).
  - ¿Está en caché?
    - **No** → GET normal → `200 OK`; se guarda con su Last-Modified y la hora de llegada.
    - **Sí** → ¿es fresca?
      - **Fresca** (edad < `max-age` y no es `no-cache`) → se sirve de inmediato, **sin ningún mensaje**.
      - **No fresca** → GET condicional (`If-Modified-Since`) → 304: vuelve a ser fresca; 200: se sustituye la copia.
  - Sin `max-age` ni `Expires`, el navegador estima la frescura (heurística: ≈10 % del tiempo desde `Last-Modified`).
    - En los ejercicios la frescura es siempre explícita.

- ¿Qué conviene guardar y durante cuánto tiempo?
  - Depende del contenido; lo fija el servidor con `max-age`:
    - Muy dinámico (portada de un periódico, páginas personalizadas, APIs) → `no-store, no-cache` o `max-age` de segundos.
    - Estático o casi (imágenes, CSS, JS, logos) → `max-age` largo: horas o días; hasta un año si el nombre lleva versión (logo.v3.png).
    - El HTML de una página suele ser lo menos cacheable; sus imágenes, lo más.
  - Cabeceras de referencia:
    - Frescura: `Cache-Control` (`max-age, no-cache, no-store`) y `Expires`.
    - Validación:
      - `Last-Modified` / `If-Modified-Since`
      - `ETag` / `If-None-Match`.

- HTTPS, HTTP/2 y HTTP/3: lo que cambia y lo que no
  - **HTTPS**: el mismo HTTP con TLS debajo (puerto 443). Cabeceras, cookies, GET condicional y caché del navegador funcionan igual. Cambia: Wireshark solo ve TLS, y un proxy no puede cachear ni filtrar sin instalar su certificado en los equipos.
  - **HTTP/2**: una conexión por servidor, varias peticiones a la vez (multiplexación) y respuestas en cualquier orden.
  - **HTTP/3**: sobre QUIC (UDP); elimina el bloqueo en cabeza de línea también en el transporte.
  - Para acercar el contenido al usuario: CDN (Redes de Distribución de Contenidos)
  - En los ejercicios y en la PEI2: HTTP/1.1 persistente, sin y con canalización